

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 21.Б07-мм

Динамическая бинарная трансляция в RISC-V с помощью Instrew

Михайлов Илья Игоревич

Отчёт по учебной практике
в форме «Решение»

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н., Луцив Д. В.

Консультант:
инженер-исследователь, Лаборатория технологий программирования инфраструктурных
решений СПбГУ, Кутуев В. А.

Санкт-Петербург
2025

Оглавление

Введение	3
1. Постановка задачи	4
2. Обзор	5
2.1. Динамические бинарные трансляторы	5
2.2. Динамические бинарные трансляторы с поднятием в LLVM IR	6
2.3. Выводы	6
3. Обзор Instrew	8
3.1. Архитектура	8
3.2. Особенности реализации Instrew	9
4. Реализация поддержки клиента Instrew	10
4.1. Компиляция	10
4.2. Исправление runtime ошибок	11
5. Реализация CI	13
6. Тестирование	14
Заключение	15
Список литературы	16

Введение

Бинарная трансляция — форма трансляции, где на вход подается последовательность инструкций одной архитектуры процессора, а на выходе получаем инструкции другой архитектуры. Такая трансляция может осуществляться с целью эмулировать какой-либо набор инструкций, чтобы запускать программы, скомпилированные не под назначенную архитектуру, либо же скомпилированные под другое расширение набора команд (важный пример — расширения RISC-V ISA). Примерами эмуляторов являются: QEMU¹, Rosetta 2, Box64². Также исходные инструкции транслировать в инструкции той же архитектуры с помощью инструментов бинарного анализа для выполнения оптимизаций и внесения патчей, например, когда рекомпиляция/декомпиляция осложнена; для профилировки и отладки транслируемой программы. Примерами таких инструментов являются Valgrind³, Pin, Dynamo.

Бинарную трансляцию можно классифицировать по способу выполнения на статическую и динамическую. Также бинарные трансляторы могут использовать поднятие инструкций до некоторого более высокоуровневого промежуточного представления. QEMU использует TCG, Valgrind использует VEX. Но хочется использовать более распространенное промежуточное представление, позволяющее делать качественные оптимизации. Одним из таких является LLVM IR.

Хочется иметь динамический бинарный транслятор с поднятием в LLVM IR у которого есть поддержка архитектуры RISC-V, как хоста.

¹<https://www.qemu.org/>

²<https://box86.org/>

³<https://valgrind.org/>

1. Постановка задачи

Целью работы является добавление поддержки архитектуры RISC-V для ДБТ Instrew. Для ее выполнения были поставлены следующие задачи:

1. выполнить обзор архитектуры Instrew;
2. реализовать минимальную функциональность для запуска Instrew на RISC-V;
3. протестировать Instrew на простых программах.

2. Обзор

Для обзора предлагается рассмотреть популярные эмуляторы и инструменты для бинарного анализа, а также отдельно рассмотреть динамические бинарные трансляторы с поднятием инструкций в LLVM IR. В данной работе не будет рассмотрен динамический транслятор Rosetta 2, так как не имеет поддержки RISC-V и не является программным обеспечением с открытым исходным кодом, несмотря на то, что существуют попытки реверс-инжиниринга⁴.

2.1. Динамические бинарные трансляторы

2.1.1. Эмуляторы

QEMU — динамический бинарный транслятор с открытым исходным кодом, предназначенный для эмуляции обширного числа архитектур, который также поддерживает аппаратную виртуализацию. В качестве промежуточного представления используется TCG, в которое поднимаются базовые блоки транслируемой программы[1]. Транслятор, исходный код и транслируемый код находятся в одном адресном пространстве во время исполнения[2]. Поддерживает RISC-V как хост-архитектуру и как целевую архитектуру.

Box64 — динамический бинарный транслятор с открытым исходным кодом, позволяющий запускать x86/x86_64 программы на архитектурах ARM и RISC-V⁵. Использует JIT-компилятор dynarec, который транслирует базовые блоки в хост архитектуру⁶. Замеры с помощью nbench показали, что box64 работает быстрее QEMU⁷.

2.1.2. Инструменты для бинарного анализа

Valgrind — инструмент для динамического бинарного анализа с открытым исходным кодом и большим числом плагинов/динамических

⁴<https://ffri.github.io/ProjectChampollion/>

⁵<https://box86.org/2024/08/box64-and-risc-v-in-2024/>

⁶<https://box86.org/2024/07/revisiting-the-dynarec/>

⁷<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10077985>

анализаторов, такие как: Memcheck, Callgrind, Cachegrind. В качестве промежуточного представления используется VEX, в которое поднимаются суперблоки транслируемой программы, где суперблок — последовательность инструкций, в котором может быть несколько точек выхода[4]. Транслятор, плагин и транслируемая программа находятся в одном адресном пространстве. Поддерживает архитектуру RISC-V.

2.2. Динамические бинарные трансляторы с поднятием в LLVM IR

2.2.1. Instrew

Instrew — динамический бинарный транслятор с поднятием инструкций в LLVM IR, открытым исходным кодом и клиент-сервер архитектурой. Как хост архитектуру процессора поддерживает x86_64 и Aarch64, как целевую — x86_64, Aarch64, RISC-V. С помощью бинарного лифтера Rellume поднимает функции в LLVM IR. Может использоваться как эмулятор, так и как инструмент для бинарного анализа кода[2].

2.2.2. DBILL

DBILL — инструмент для динамического бинарного анализа, использующий TCG и LLVM IR, как промежуточные представления. Благодаря фронтэнду QEMU и тулчейну LLVM поддерживается множество хост и целевых архитектур⁸. Из-за двух промежуточных представлений и поднятия в TCG только базовых блоков, есть проблемы с производительностью. Исходный код не был найден.

2.3. Выводы

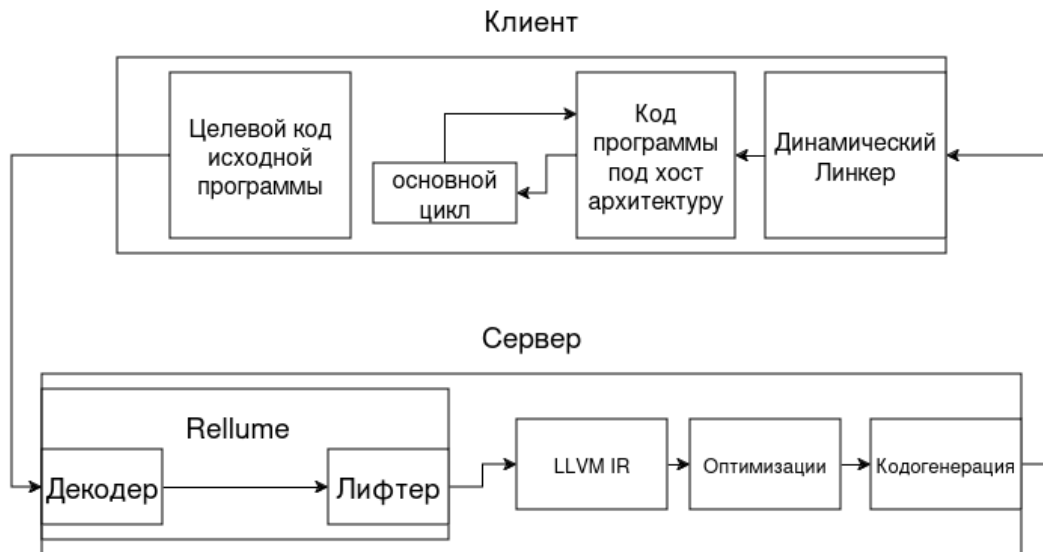
По результатам замеров[3], Instrew оказался более производительным, чем QEMU и Valgrind. Хотя Box64 тоже оказывается производительнее чем QEMU, но не имеет API для произведения динамического

⁸<https://doi.org/10.1145/2674025.2576213>

бинарного анализа, что в сравнении с Instrew делает его менее универсальным. Далее данная работа будет сосредоточена именно на ДБТ Instrew с поднятием в LLVM IR и добавлении поддержки архитектуры RISC-V для такого транслятора.

3. Обзор Instrew

3.1. Архитектура



Instrew реализует клиент-сервер архитектуру:

- клиент (также instrew-client), написанный на языке C и ассемблере, и выполняющий следующие функции:
 - загрузка целевой программы в адресное пространство;
 - эмуляция состояния процессора целевой программы;
 - отправка функций целевой программы серверу;
 - динамическая линковка ELF объектов, полученных от сервера;
 - загрузка, разрешение символов и проведение релокаций для полученного ELF объекта;
 - эмуляция системных вызовов;
 - вызов функции, полученной после линковки ELF объекта;
 - кэширование;
- сервер, написанный на языке C++, выполняющий функции:
 - декодирование и поднятие функции целевой программы в LLVM IR с помощью Rellume;

- оптимизация LLVM IR;
- кодогенерация;
- создание ELF объекта;

3.2. Особенности реализации Instrew

4. Реализация поддержки клиента Instrew

4.1. Компиляция

Для начала необходимо скомпилировать Instrew, а именно:

- проставить константы и макросы;
- написать точку входа в программу на ассемблере;
- написать реализацию `syscallN` для собственной реализации `libc`, т.н. `minilibc`.

Из трудностей можно выделить, что при написании точки входа в программу был просмотрен код канонических реализаций `libc` (в `glibc` — файл `start.S`, в `musl` — `crt_arch.S`), где была обнаружена плохо задокументированная инструкция `lla`⁹.

Листинг 1: Заголовок

```
ASM_BLOCK(  
    .weak _DYNAMIC;  
    .hidden _DYNAMIC;  
    .global _start;  
    _start:  
        mov x0, sp;  
        adrp x1, _DYNAMIC;  
        add x1, x1, #:lo12:_DYNAMIC;  
        and sp, x0, #0xfffffffffffff0;  
        bl __start_main;  
);
```

Листинг 2: Системный вызов

```
static size_t syscall2(int n, size_t a, size_t b)  
{  
    register size_t a7 __asm__("a7") = n;  
    register size_t a0 __asm__("a0") = a;  
    register size_t a1 __asm__("a1") = b;  
    __asm__ volatile ("ecall\n\t" : "+r"(a0) : "r"(a7), "0"(a0), "r"(a1) : "  
        memory");  
    return a0;  
}
```

⁹<https://reviews.llvm.org/D49661>

4.2. Исправление runtime ошибок

После успешной компиляции пробуем запустить instrew. В данном разделе хочется рассказать о трудностях отладки клиента Instrew и [способах борьбы с ними].

4.2.1. Специфические флаги сборки

При запуске клиента instrew без аргументов получаем ошибку сегментации. Пытаемся отладить программу с помощью gdb. Asan и gprof подключить не удалось, из-за конфликтующих флагов сборки, а именно:

- флаги в asan;
- флаг -pg.

После отладки с помощью gdb и просмотра memory mappings, оказалось, что семантика флага **-static-pie** при компиляции с помощью gcc-13 отличается на платформах x86 и riscv, а именно:

- подключается динамический линкер ld-linux;
- но клиент Instrew сам является динамическим линкером и сам обрабатывает свои релокации.

Из-за этого происходит ошибка сегментации, так как клиент Instrew пытается писать в зарезервированную ld-linux память.

4.2.2. Связь атрибутов и релокаций

Далее также при запуске клиента instrew без аргументов программа выходит с exit code -ENOEXEC. Оказалось, что это происходит при обработке одной из релокаций. Оказалось что над функцией **memset** стоит атрибут weak, из-за чего в итоге меняется тип релокации. Но эту релокацию instrew не может обработать.

4.2.3. Дебаг программ, использующих `fork()` и `execve()`

После успешного запуска клиента `instrew`, пробуем запустить сервер `instrew`. Чтобы можно было пройтись по программе в `gdb`, необходимо понимать как вызывается `instrew-client`:

1. делается `fork()` процесса, где дочерний процесс является клиентом `instrew`;
2. в дочернем процессе вызывается `execve`, который запускает программу `instrew-client`.

4.2.4. Релокации и PLT-трамплины

Листинг 3: Релокация

```
uint64_t syma = (uintptr_t) sym + patch_data->addend;
uint64_t pc = patch_data->patch_addr;
int64_t prel_syma = syma - (int64_t) pc;

case R_RISCV_32_PCREL:

    if (!rtld_elf_signed_range(prel_syma, 32, "R_RISCV_32_PCREL"))
        return -EINVAL;
    rtld_blend(tgt, 0xffffffff, prel_syma);
    break;
```

5. Реализация СИ

6. Тестирование

- Удалось успешно протестировать на:
 - программах без libc, исполняющих системные вызовы;
 - факториале и фибоначчи, выводящих результат в exit code;
 - факториале с минимальным рантаймом, который выводит результат на экран.
- Не удалось протестировать на:
 - динамически слинкованных программах и программах использующих libc;
 - программах, с флагами сборки -pie;
 - программах, где конкретная релокация еще не реализована.
- Вывод:
 - работают статически слинкованные программы, с простым рантаймом и системными вызовами

Заключение

В результате работы были выполнены следующие задачи:

1. выполнен обзор архитектуры Instrew;
2. реализованы процессорно-специфические патчи и проставлены константы;
3. реализована эмуляция системных вызовов, PLT трамплины и минимальный набор ELF релокаций;
4. Instrew был протестирован на статически слинкованных программах, с простым рантаймом.

Код доступен по данной [ссылке](#).

Список литературы

- [1] Bellard Fabrice. QEMU, a fast and portable dynamic translator // Proceedings of the Annual Conference on USENIX Annual Technical Conference. — ATEC '05. — USA : USENIX Association, 2005. — P. 41.
- [2] Engelke Alexis, Schulz Martin. [Instrew: leveraging LLVM for high performance dynamic binary instrumentation](#) // Proceedings of the 16th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. — VEE '20. — New York, NY, USA : Association for Computing Machinery, 2020. — P. 172–184. — URL: <https://doi.org/10.1145/3381052.3381319>.
- [3] Engelke Alexis Friedrich. Optimizing Performance Using Dynamic Code Generation : Ph. D. thesis / Alexis Friedrich Engelke ; Technische Universität München. — 2021. — P. 163. — URL: <https://mediatum.ub.tum.de/1614897>.
- [4] Nethercote Nicholas, Seward Julian. [Valgrind: a framework for heavy-weight dynamic binary instrumentation](#) // Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation. — PLDI '07. — New York, NY, USA : Association for Computing Machinery, 2007. — P. 89–100. — URL: <https://doi.org/10.1145/1250734.1250746>.